

# **Restaurant Management System**

## System Design Document

Nashya K A — S5 CSE  
Saniya Mansoor — S5 CSE  
Sufiyan Shiraj Mohammed — S3 CSE  
Surya Nandini — S5 CSE

**Team Zenith**

June 16, 2026

# Contents

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| <b>1</b> | <b>Architecture Overview</b>                             | <b>4</b>  |
| 1.1      | Client Layer . . . . .                                   | 4         |
| 1.2      | Application Layer . . . . .                              | 4         |
| 1.3      | Data Layer . . . . .                                     | 4         |
| 1.4      | Deployment . . . . .                                     | 4         |
| <b>2</b> | <b>Tech Stack</b>                                        | <b>6</b>  |
| 2.1      | Justification . . . . .                                  | 6         |
| <b>3</b> | <b>Database Schema</b>                                   | <b>7</b>  |
| 3.1      | profiles . . . . .                                       | 7         |
| 3.2      | categories . . . . .                                     | 7         |
| 3.3      | menu_items . . . . .                                     | 7         |
| 3.4      | tables . . . . .                                         | 8         |
| 3.5      | table_sessions . . . . .                                 | 8         |
| 3.6      | session_devices . . . . .                                | 8         |
| 3.7      | orders . . . . .                                         | 9         |
| 3.8      | order_items . . . . .                                    | 9         |
| 3.9      | shifts . . . . .                                         | 9         |
| 3.10     | shift_switch_requests . . . . .                          | 10        |
| 3.11     | surplus_items . . . . .                                  | 10        |
| 3.12     | Table Relationships . . . . .                            | 11        |
| <b>4</b> | <b>API Routes</b>                                        | <b>12</b> |
| 4.1      | Authentication — handled by Supabase Auth . . . . .      | 12        |
| 4.2      | Categories — /api/categories . . . . .                   | 12        |
| 4.3      | Menu Items — /api/menu . . . . .                         | 12        |
| 4.4      | Tables — /api/tables . . . . .                           | 12        |
| 4.5      | Table Sessions — /api/sessions . . . . .                 | 13        |
| 4.6      | Orders — /api/orders . . . . .                           | 13        |
| 4.7      | Shifts — /api/shifts . . . . .                           | 14        |
| 4.8      | Shift Switch Requests — /api/shifts/requests . . . . .   | 14        |
| 4.9      | Users — /api/users . . . . .                             | 14        |
| 4.10     | Surplus — /api/surplus . . . . .                         | 14        |
| 4.11     | Route Protection Pattern . . . . .                       | 14        |
| <b>5</b> | <b>Component Structure</b>                               | <b>16</b> |
| <b>6</b> | <b>Authentication Flow</b>                               | <b>18</b> |
| 6.1      | Admin and Staff Login . . . . .                          | 18        |
| 6.2      | Customer Access via QR . . . . .                         | 18        |
| 6.3      | Route Protection . . . . .                               | 18        |
| <b>7</b> | <b>Realtime Architecture</b>                             | <b>19</b> |
| 7.1      | Table Session Page — Customer Ordering . . . . .         | 19        |
| 7.2      | Kitchen Display — Live Order Updates . . . . .           | 19        |
| 7.3      | Wait Time Estimation — Real-Time Recalculation . . . . . | 20        |

|           |                                         |           |
|-----------|-----------------------------------------|-----------|
| 7.4       | Bill Trigger — Complete Table . . . . . | 21        |
| 7.5       | Shift Switch Notifications . . . . .    | 22        |
| <b>8</b>  | <b>Data Flow</b>                        | <b>23</b> |
| <b>9</b>  | <b>Security</b>                         | <b>27</b> |
| 9.1       | Authentication Security . . . . .       | 27        |
| 9.2       | Authorization — Three Layers . . . . .  | 27        |
| 9.3       | Environment Variables . . . . .         | 27        |
| 9.4       | Known Limitations . . . . .             | 27        |
| <b>10</b> | <b>Future Scope</b>                     | <b>28</b> |

# 1 Architecture Overview

The system follows a three-tier architecture: a client layer, an application layer, and a data layer. All three communicate over HTTPS.

## 1.1 Client Layer

Users interact with the system through a web browser on any device. Next.js serves all pages and handles UI rendering. Customer-facing pages are publicly accessible via table QR codes. Staff and Admin pages are protected by session-based role checks. The kitchen display is a dedicated staff-facing view optimised for large screens.

## 1.2 Application Layer

Next.js handles both the frontend and the backend in a single project. Page routing is handled by the App Router. Backend logic is handled by Next.js API Route Handlers defined inside `app/api/`. These routes receive requests, validate the user session and role, and communicate with Supabase.

## 1.3 Data Layer

Supabase provides a PostgreSQL database, built-in authentication, Row Level Security, and the Realtime engine. Realtime subscriptions are used across multiple modules to push live updates to connected clients without polling.

## 1.4 Deployment

The Next.js application is deployed on Vercel. Supabase is a separate cloud-hosted service. Both connect via environment variables. Git is used for version control. All changes are submitted via pull requests before merging to main.

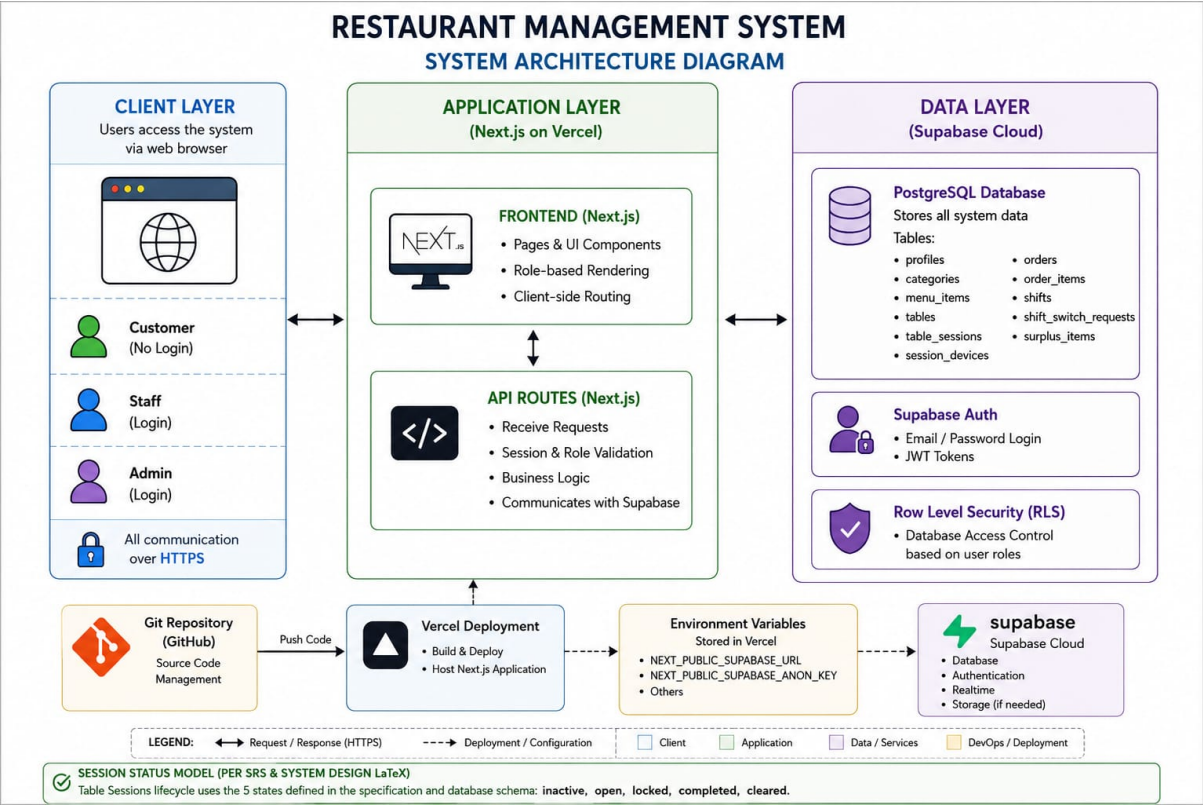


Figure 1: System Architecture Diagram

## 2 Tech Stack

| Layer           | Technology             | Purpose                               |
|-----------------|------------------------|---------------------------------------|
| Frontend        | Next.js 14+ App Router | Pages, routing, UI rendering          |
| Backend         | Next.js API Routes     | Server-side logic and role validation |
| Database        | Supabase (PostgreSQL)  | Data storage, auth, and realtime      |
| Language        | JavaScript             | Primary development language          |
| Styling         | Tailwind CSS           | UI styling and layout                 |
| Scheduling UI   | @fullcalendar/react    | Drag and drop staff timetable         |
| Version Control | Git                    | Code management and collaboration     |
| Deployment      | Vercel                 | Hosting and production deployment     |

Table 1: Tech Stack

### 2.1 Justification

**Next.js** combines frontend and backend in one project removing the need for a separate server. The App Router is the current industry standard and is natively optimised for Vercel.

**Supabase** provides PostgreSQL, built-in auth, Row Level Security, and a Realtime engine in one platform. The free tier covers the full scope of this project.

**Tailwind CSS** allows styling to be written directly in component files removing the need for separate stylesheets per component.

**@fullcalendar/react** provides a production-ready time grid with built-in drag and drop support. It requires less manual configuration than alternatives like react-big-calendar and has better Next.js compatibility.

**Vercel** is built by the same team as Next.js. Deployments trigger automatically on every Git push.

### 3 Database Schema

The system uses eleven tables in a Supabase PostgreSQL database. All primary keys use UUID. All timestamps use `timestamptz` for timezone awareness.

#### 3.1 profiles

Extends Supabase Auth. Created automatically when a new auth user registers via a database trigger.

| Column     | Type        | Required | Notes                                           |
|------------|-------------|----------|-------------------------------------------------|
| id         | uuid        | Yes      | References auth.users(id), cascades on delete   |
| full_name  | text        | Yes      | Display name                                    |
| role       | text        | Yes      | Either <code>admin</code> or <code>staff</code> |
| created_at | timestamptz | Yes      | Auto-generated                                  |

Table 2: profiles

#### 3.2 categories

| Column     | Type        | Required | Notes                               |
|------------|-------------|----------|-------------------------------------|
| id         | uuid        | Yes      | Auto-generated primary key          |
| name       | text        | Yes      | Category name e.g. Starters, Drinks |
| created_at | timestamptz | Yes      | Auto-generated                      |

Table 3: categories

#### 3.3 menu\_items

| Column            | Type          | Required | Notes                                               |
|-------------------|---------------|----------|-----------------------------------------------------|
| id                | uuid          | Yes      | Auto-generated primary key                          |
| name              | text          | Yes      | Item name                                           |
| price             | numeric(10,2) | Yes      | Price, default 0.00, must be $\geq 0$               |
| category_id       | uuid          | Yes      | References categories(id), restrict on delete       |
| is_available      | boolean       | Yes      | Daily availability toggle, default true             |
| is_archived       | boolean       | Yes      | Hides item from menu entirely, default false        |
| prep_time_minutes | integer       | Yes      | Admin-set prep time for wait estimation, default 15 |
| created_at        | timestamptz   | Yes      | Auto-generated                                      |

Table 4: menu\_items

### 3.4 tables

| Column       | Type       | Required | Notes                                  |
|--------------|------------|----------|----------------------------------------|
| id           | uuid       | Yes      | Auto-generated primary key             |
| table_number | integer    | Yes      | Unique table identifier                |
| capacity     | integer    | Yes      | Number of seats at this table          |
| qr_code_url  | text       | Yes      | Static URL embedded in printed QR code |
| created_at   | timestampz | Yes      | Auto-generated                         |

Table 5: tables

### 3.5 table\_sessions

One record per seating. Sessions are never deleted — they are archived when closed.

| Column       | Type          | Required | Notes                                                     |
|--------------|---------------|----------|-----------------------------------------------------------|
| id           | uuid          | Yes      | Auto-generated primary key                                |
| table_id     | uuid          | Yes      | References tables(id)                                     |
| pin          | char(4)       | Yes      | 4-digit PIN given to customers verbally                   |
| status       | text          | Yes      | One of: inactive, open, locked, completed, cleared        |
| opened_at    | timestampz    | No       | Set when session is created                               |
| completed_at | timestampz    | No       | Set when Complete Table is triggered                      |
| cleared_at   | timestampz    | No       | Set when session is fully cleared                         |
| closed_by    | uuid          | No       | References profiles(id), who cleared the session          |
| total_amount | numeric(10,2) | Yes      | Running total, updated as orders are placed, default 0.00 |
| created_at   | timestampz    | Yes      | Auto-generated                                            |

Table 6: table\_sessions

### 3.6 session\_devices

Tracks every device that joined a table session for log purposes.

| Column             | Type       | Required | Notes                                        |
|--------------------|------------|----------|----------------------------------------------|
| id                 | uuid       | Yes      | Auto-generated primary key                   |
| session_id         | uuid       | Yes      | References table_sessions(id)                |
| device_fingerprint | text       | Yes      | Browser-generated ID stored in local-Storage |
| joined_at          | timestampz | Yes      | Auto-generated on insert                     |

Table 7: session\_devices



### 3.7 orders

| Column                 | Type       | Required | Notes                                                                  |
|------------------------|------------|----------|------------------------------------------------------------------------|
| id                     | uuid       | Yes      | Auto-generated primary key                                             |
| session_id             | uuid       | Yes      | References table.sessions(id)                                          |
| status                 | text       | Yes      | One of: placed, preparing, ready, delivered, cancelled. Default placed |
| estimated_wait_minutes | integer    | No       | Calculated on order placement, updated as kitchen load changes         |
| rating                 | integer    | No       | Customer rating 1-5, null until submitted                              |
| created_at             | timestampz | Yes      | Auto-generated                                                         |

Table 8: orders

### 3.8 order\_items

| Column          | Type          | Required | Notes                                              |
|-----------------|---------------|----------|----------------------------------------------------|
| id              | uuid          | Yes      | Auto-generated primary key                         |
| order_id        | uuid          | Yes      | References orders(id), cascades on delete          |
| menu_item_id    | uuid          | Yes      | References menu.items(id), restrict on delete      |
| quantity        | integer       | Yes      | Must be $\geq 0$ , default 1                       |
| price_at_order  | numeric(10,2) | Yes      | Price snapshot at time of order, must be $\geq 0$  |
| item_status     | text          | Yes      | One of: pending, preparing, ready. Default pending |
| item_started_at | timestampz    | No       | Set when staff taps Start on this item             |
| created_at      | timestampz    | Yes      | Auto-generated                                     |

Table 9: order\_items

### 3.9 shifts

| Column     | Type       | Required | Notes                      |
|------------|------------|----------|----------------------------|
| id         | uuid       | Yes      | Auto-generated primary key |
| staff_id   | uuid       | Yes      | References profiles(id)    |
| station    | text       | Yes      | e.g. Kitchen, Floor, Bar   |
| start_time | timestampz | Yes      | Shift start                |
| end_time   | timestampz | Yes      | Shift end                  |
| created_at | timestampz | Yes      | Auto-generated             |

Table 10: shifts

### 3.10 shift\_switch\_requests

| Column             | Type       | Required | Notes                                                                |
|--------------------|------------|----------|----------------------------------------------------------------------|
| id                 | uuid       | Yes      | Auto-generated primary key                                           |
| requester_id       | uuid       | Yes      | Staff member requesting the switch, references profiles(id)          |
| target_id          | uuid       | Yes      | Staff member whose shift is being requested, references profiles(id) |
| requester_shift_id | uuid       | Yes      | References shifts(id)                                                |
| target_shift_id    | uuid       | Yes      | References shifts(id)                                                |
| status             | text       | Yes      | One of: pending, approved, rejected. Default pending                 |
| created_at         | timestampz | Yes      | Auto-generated                                                       |

Table 11: shift\_switch\_requests

### 3.11 surplus\_items

| Column              | Type          | Required | Notes                                                |
|---------------------|---------------|----------|------------------------------------------------------|
| id                  | uuid          | Yes      | Auto-generated primary key                           |
| menu_item_id        | uuid          | Yes      | References menu.items(id)                            |
| quantity            | integer       | Yes      | Portions available, must be $\geq 0$                 |
| is_free             | boolean       | Yes      | Whether item is free, default false                  |
| discounted_price    | numeric(10,2) | Yes      | Price of the item, can be set to 0.00 for free items |
| pickup_window_start | timestampz    | Yes      | Start of public pickup window                        |
| pickup_window_end   | timestampz    | Yes      | End of public pickup window                          |
| is_claimed          | boolean       | Yes      | Admin marks when collected, default false            |
| total_given_away    | integer       | Yes      | Running count of portions given, default 0           |
| created_at          | timestampz    | Yes      | Auto-generated                                       |

Table 12: surplus\_items

### 3.12 Table Relationships

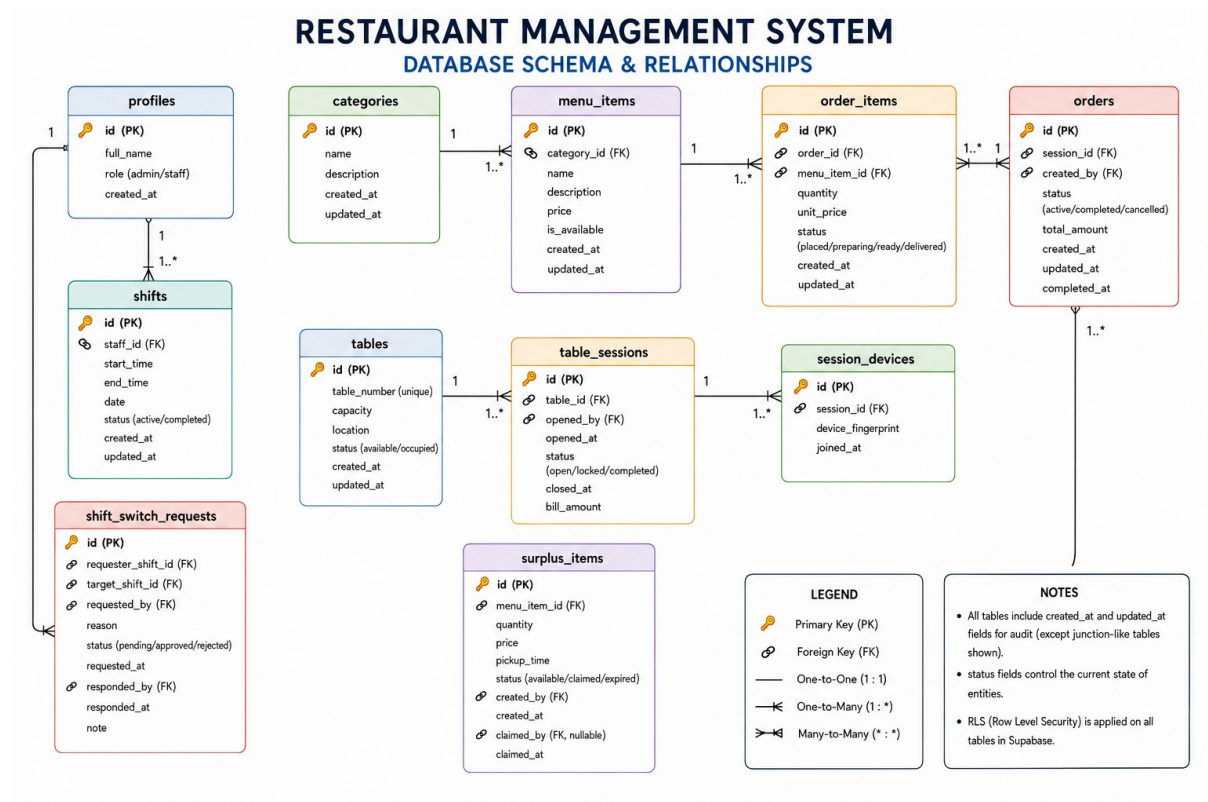


Figure 2: Database Schema and Relationships

## 4 API Routes

All routes live inside `app/api/`. Protected routes validate the session token and role before executing. Customer-facing routes (orders, table session) validate the session device fingerprint instead.

### 4.1 Authentication — handled by Supabase Auth

No custom API routes needed for login or logout.

### 4.2 Categories — `/api/categories`

| Method | Endpoint                          | Access | Description          |
|--------|-----------------------------------|--------|----------------------|
| GET    | <code>/api/categories</code>      | All    | Fetch all categories |
| POST   | <code>/api/categories</code>      | Admin  | Create category      |
| PUT    | <code>/api/categories/[id]</code> | Admin  | Edit category        |
| DELETE | <code>/api/categories/[id]</code> | Admin  | Delete category      |

Table 13: Category routes

### 4.3 Menu Items — `/api/menu`

| Method | Endpoint                        | Access       | Description                                 |
|--------|---------------------------------|--------------|---------------------------------------------|
| GET    | <code>/api/menu</code>          | All          | Fetch all non-archived items                |
| GET    | <code>/api/menu?all=true</code> | Admin, Staff | Fetch all items including archived          |
| POST   | <code>/api/menu</code>          | Admin        | Add new item                                |
| PUT    | <code>/api/menu/[id]</code>     | Admin, Staff | Edit item or toggle availability or archive |
| DELETE | <code>/api/menu/[id]</code>     | Admin        | Delete item                                 |

Table 14: Menu routes

### 4.4 Tables — `/api/tables`

| Method | Endpoint                      | Access       | Description                                  |
|--------|-------------------------------|--------------|----------------------------------------------|
| GET    | <code>/api/tables</code>      | Admin, Staff | Fetch all tables with current session status |
| POST   | <code>/api/tables</code>      | Admin        | Add new table                                |
| PUT    | <code>/api/tables/[id]</code> | Admin        | Edit table details                           |
| DELETE | <code>/api/tables/[id]</code> | Admin        | Delete table                                 |

Table 15: Table routes

## 4.5 Table Sessions — /api/sessions

| Method | Endpoint                    | Access       | Description                                   |
|--------|-----------------------------|--------------|-----------------------------------------------|
| GET    | /api/sessions               | Admin        | Fetch session history log                     |
| GET    | /api/sessions/[id]          | Admin, Staff | Fetch single session details                  |
| POST   | /api/sessions               | Admin, Staff | Create new session, generates PIN             |
| PUT    | /api/sessions/[id]/complete | Admin, Staff | Trigger Complete Table, sends bill to devices |
| PUT    | /api/sessions/[id]/clear    | Admin, Staff | Clear session, log closed by and cleared at   |
| POST   | /api/sessions/[id]/join     | Customer     | Validate PIN and register device fingerprint  |

Table 16: Session routes

## 4.6 Orders — /api/orders

| Method | Endpoint                        | Access                 | Description                   |
|--------|---------------------------------|------------------------|-------------------------------|
| GET    | /api/orders                     | Admin, Staff           | Fetch all orders              |
| GET    | /api/orders/[id]                | Admin, Staff, Customer | Fetch single order            |
| POST   | /api/orders                     | Customer               | Place new order from session  |
| PUT    | /api/orders/[id]/status         | Admin, Staff           | Update full order status      |
| PUT    | /api/orders/[id]/items/[itemId] | Admin, Staff           | Update individual item status |

Table 17: Order routes

## 4.7 Shifts — /api/shifts

| Method | Endpoint         | Access       | Description                    |
|--------|------------------|--------------|--------------------------------|
| GET    | /api/shifts      | Admin, Staff | Fetch all shifts for timetable |
| POST   | /api/shifts      | Admin        | Create new shift block         |
| PUT    | /api/shifts/[id] | Admin        | Edit shift time or station     |
| DELETE | /api/shifts/[id] | Admin        | Delete shift                   |

Table 18: Shift routes

## 4.8 Shift Switch Requests — /api/shifts/requests

| Method | Endpoint                  | Access | Description                |
|--------|---------------------------|--------|----------------------------|
| GET    | /api/shifts/requests      | Admin  | Fetch all pending requests |
| POST   | /api/shifts/requests      | Staff  | Submit switch request      |
| PUT    | /api/shifts/requests/[id] | Admin  | Approve or reject request  |

Table 19: Shift switch request routes

## 4.9 Users — /api/users

| Method | Endpoint        | Access       | Description                 |
|--------|-----------------|--------------|-----------------------------|
| GET    | /api/users      | Admin        | Fetch all accounts          |
| POST   | /api/users      | Admin        | Create new account          |
| PUT    | /api/users/[id] | Admin        | Edit account or change role |
| PUT    | /api/users/me   | Admin, Staff | Edit own profile            |
| DELETE | /api/users/[id] | Admin        | Delete account              |

Table 20: User routes

## 4.10 Surplus — /api/surplus

| Method | Endpoint          | Access | Description                                                     |
|--------|-------------------|--------|-----------------------------------------------------------------|
| GET    | /api/surplus      | Public | Fetch active surplus items for public forum (no login required) |
| POST   | /api/surplus      | Admin  | Mark items as surplus                                           |
| PUT    | /api/surplus/[id] | Admin  | Mark as claimed or update details                               |
| DELETE | /api/surplus/[id] | Admin  | Remove surplus listing                                          |

Table 21: Surplus routes

## 4.11 Route Protection Pattern

Every protected route follows this logic:

1. Check for valid Supabase session token
2. If no session — return **401 Unauthorized**
3. Check role matches route requirement
4. If role mismatch — return **403 Forbidden**
5. Execute database query and return result

Customer routes validate the device fingerprint against `session_devices` instead of a session token.

## 5 Component Structure

|                              |                                                     |
|------------------------------|-----------------------------------------------------|
| app/                         |                                                     |
| +-- page.jsx                 | <i>Welcome page</i>                                 |
| +-- login/                   |                                                     |
| +-- page.jsx                 | <i>Login page for Staff and Admin</i>               |
| +-- dashboard/               |                                                     |
| +-- page.jsx                 | <i>Admin dashboard</i>                              |
| +-- menu/                    |                                                     |
| +-- page.jsx                 | <i>Menu management for Admin and Staff</i>          |
| +-- tables/                  |                                                     |
| +-- page.jsx                 | <i>Table overview for Admin and Staff</i>           |
| +-- [id]/page.jsx            | <i>Individual table detail and session view</i>     |
| +-- orders/                  |                                                     |
| +-- page.jsx                 | <i>All orders view for Admin and Staff</i>          |
| +-- kitchen/                 |                                                     |
| +-- page.jsx                 | <i>Kitchen display for Staff and Admin</i>          |
| +-- schedule/                |                                                     |
| +-- page.jsx                 | <i>Staff scheduling timetable</i>                   |
| +-- users/                   |                                                     |
| +-- page.jsx                 | <i>User management for Admin</i>                    |
| +-- surplus/                 |                                                     |
| +-- page.jsx                 | <i>Admin surplus management</i>                     |
| +-- surplus/public/          |                                                     |
| +-- page.jsx                 | <i>Public surplus listing, no login</i>             |
| +-- table/[tableId]/session/ |                                                     |
| +-- page.jsx                 | <i>Customer QR landing and session page</i>         |
| +-- api/                     | <i>All route handlers as described in Section 4</i> |
| _components/                 |                                                     |
| +-- layout/                  |                                                     |
| +-- Navbar.jsx               | <i>Top navigation bar</i>                           |
| +-- Sidebar.jsx              | <i>Role-based side navigation</i>                   |
| +-- menu/                    |                                                     |
| +-- MenuCard.jsx             | <i>Single item card with role-based controls</i>    |
| +-- MenuGrid.jsx             | <i>Category-grouped item grid</i>                   |
| +-- AddItemModal.jsx         | <i>Add or edit item popup</i>                       |
| +-- CategoryFilter.jsx       | <i>Filter items by category</i>                     |
| +-- tables/                  |                                                     |
| +-- TableCard.jsx            | <i>Single table with status badge</i>               |
| +-- TableDetailModal.jsx     | <i>Session details, devices, orders, total</i>      |
| +-- PinModal.jsx             | <i>Customer PIN entry screen</i>                    |
| +-- orders/                  |                                                     |
| +-- OrderCard.jsx            | <i>Collapsed order summary</i>                      |
| +-- OrderExpanded.jsx        | <i>Full item list with statuses</i>                 |
| +-- CartSidebar.jsx          | <i>Customer cart with total</i>                     |
| +-- kitchen/                 |                                                     |



|                            |                                                      |
|----------------------------|------------------------------------------------------|
| +++ KitchenOrderCard.jsx   | <i>Order with item-level start and ready buttons</i> |
| +++ WaitTimeDisplay.jsx    | <i>Estimated wait time per order</i>                 |
| +++ billing/               |                                                      |
| +++ BillDisplay.jsx        | <i>Full bill with all session orders</i>             |
| +++ UpiQrCode.jsx          | <i>UPI QR with pre-filled amount</i>                 |
| +++ RatingPrompt.jsx       | <i>Optional 1-5 star rating after bill</i>           |
| +++ schedule/              |                                                      |
| +++ ScheduleCalendar.jsx   | <i>FullCalendar drag and drop timetable</i>          |
| +++ SwitchRequestModal.jsx | <i>Submit or review switch request</i>               |
| +++ dashboard/             |                                                      |
| +++ StatCard.jsx           | <i>Single summary stat</i>                           |
| +++ users/                 |                                                      |
| +++ UserTable.jsx          | <i>List of all accounts</i>                          |
| +++ UserModal.jsx          | <i>Create or edit user</i>                           |
| +++ surplus/               |                                                      |
| +++ SurplusForm.jsx        | <i>Admin marks surplus items</i>                     |
| +++ SurplusCard.jsx        | <i>Public surplus item display</i>                   |
| _lib/                      |                                                      |
| +++ supabase.js            | <i>Supabase client connection</i>                    |
| +++ realtimeHelpers.js     | <i>Shared Supabase Realtime subscription setup</i>   |
| +++ waitTimeCalculator.js  | <i>Kitchen load and wait time logic</i>              |
| +++ upiHelper.js           | <i>UPI QR string generator</i>                       |
| +++ utils.js               | <i>Shared helper functions</i>                       |
| _styles/                   |                                                      |
| +++ globals.css            | <i>Global CSS styles</i>                             |

## 6 Authentication Flow

### 6.1 Admin and Staff Login

Admin and Staff authenticate using email and password via Supabase Auth. On success, Supabase issues a JWT session token stored in browser cookies. The application fetches the user role from the profiles table and stores it in application state. Admin is redirected to the Dashboard. Staff is redirected to the Orders page.

### 6.2 Customer Access via QR

Customers do not have accounts. They access the system by scanning the table QR code, which opens the table session page. They enter the 4-digit PIN given by the waiter. The system validates the PIN against the active session record. If valid, a device fingerprint is generated in the browser and registered in session\_devices. The customer gains access to the session page scoped to that table.

### 6.3 Route Protection

All Admin and Staff pages check for an active Supabase session on load. No session redirects to login. Session with wrong role redirects away. Customer pages validate the device fingerprint from localStorage against session\_devices on every load.

## 7 Realtime Architecture

Supabase Realtime is used across five key scenarios. Each uses a database subscription that pushes changes to connected clients instantly without page refresh.

### 7.1 Table Session Page — Customer Ordering

All devices connected to the same table session subscribe to the `orders` table filtered by `session_id`. When any device places an order, all other devices in the same session receive it instantly.

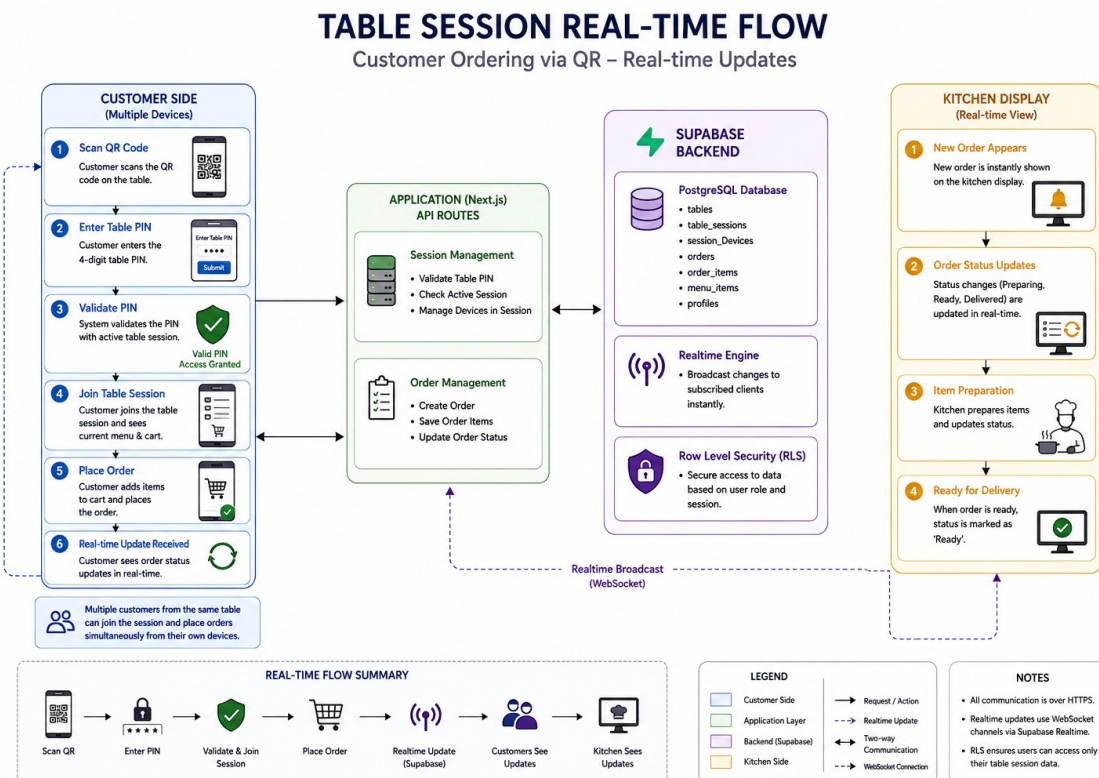


Figure 3: Table Session Real-Time Flow Diagram

### 7.2 Kitchen Display — Live Order Updates

The kitchen display subscribes to both the `orders` table and the `order_items` table. When any order status or individual item status changes, the kitchen display updates instantly.

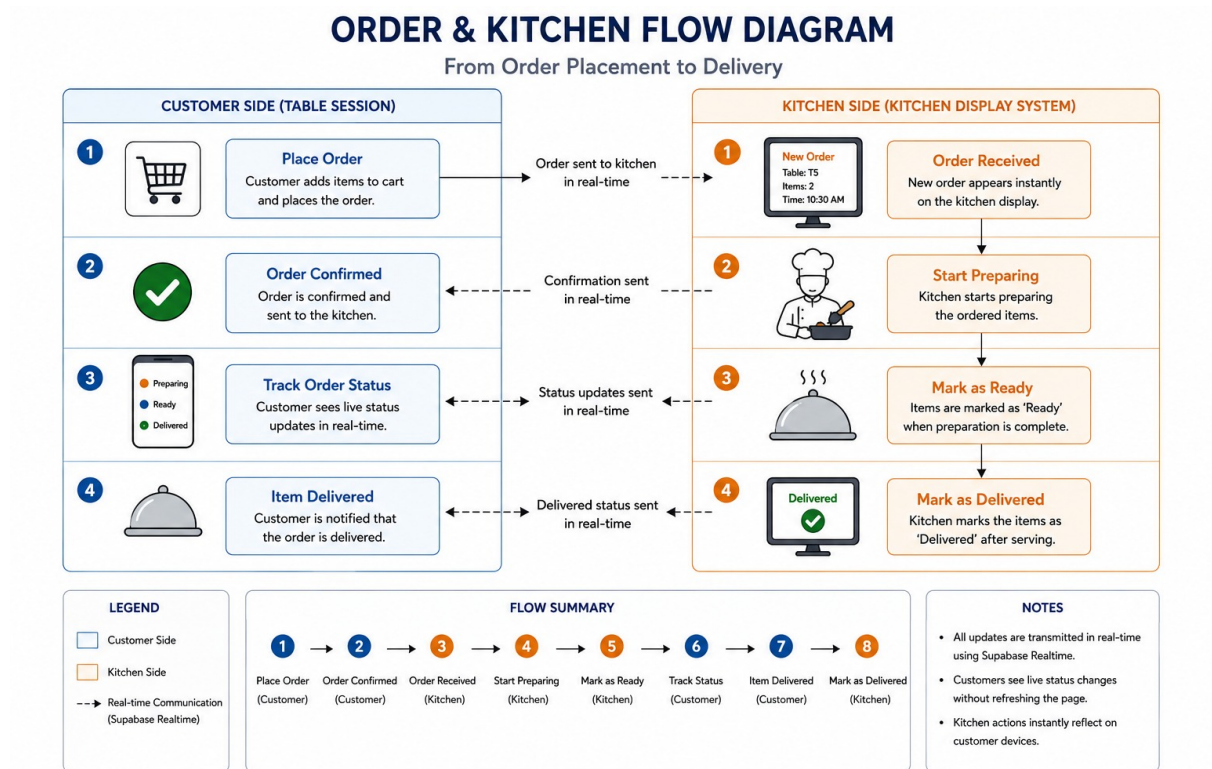


Figure 4: Order and Kitchen Flow Diagram

### 7.3 Wait Time Estimation — Real-Time Recalculation

When a new order is placed, the system calculates estimated wait time using the following logic:

1. For every item currently in **preparing** status across all active orders, calculate remaining time: `prep_time_minutes` minus elapsed time since `item_started_at`.
2. The kitchen load equals the longest remaining time across all preparing items.
3. The new order wait time equals the kitchen load plus the prep time of the longest item in the new order.
4. This value is stored in `estimated_wait_minutes` on the order record.
5. As items are marked ready, the system recalculates estimates for all remaining pending orders and updates via Realtime.

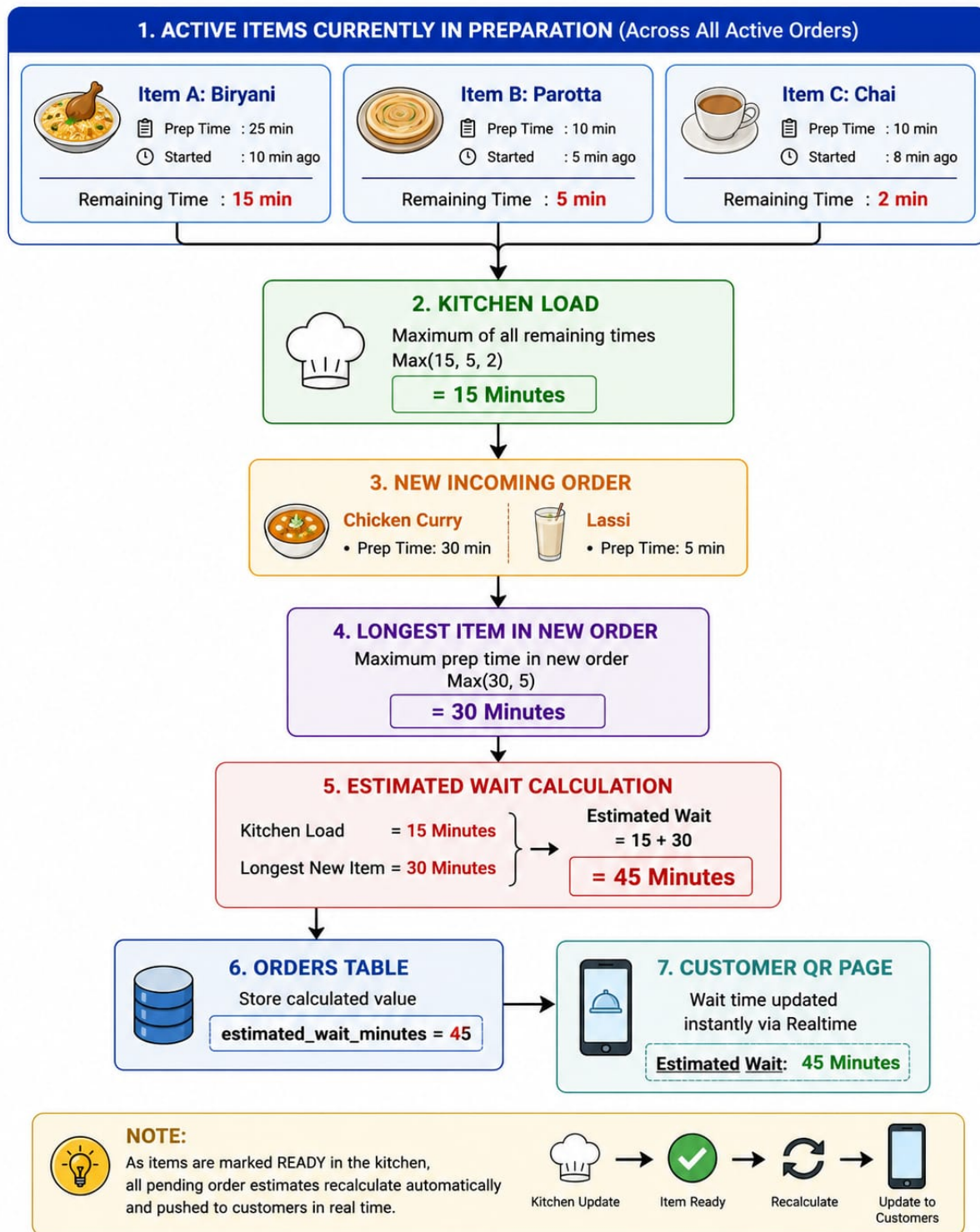


Figure 5: Wait Time Estimation Flow Diagram

## 7.4 Bill Trigger — Complete Table

When Admin or Staff triggers Complete Table, a Realtime broadcast is sent to all devices connected to that table session. All devices simultaneously receive the bill screen. No device needs to refresh.

## 7.5 Shift Switch Notifications

When a switch request is submitted, a Realtime event is sent to the Admin's dashboard and to the target staff member's session notifying them of the pending request.



## 8 Data Flow

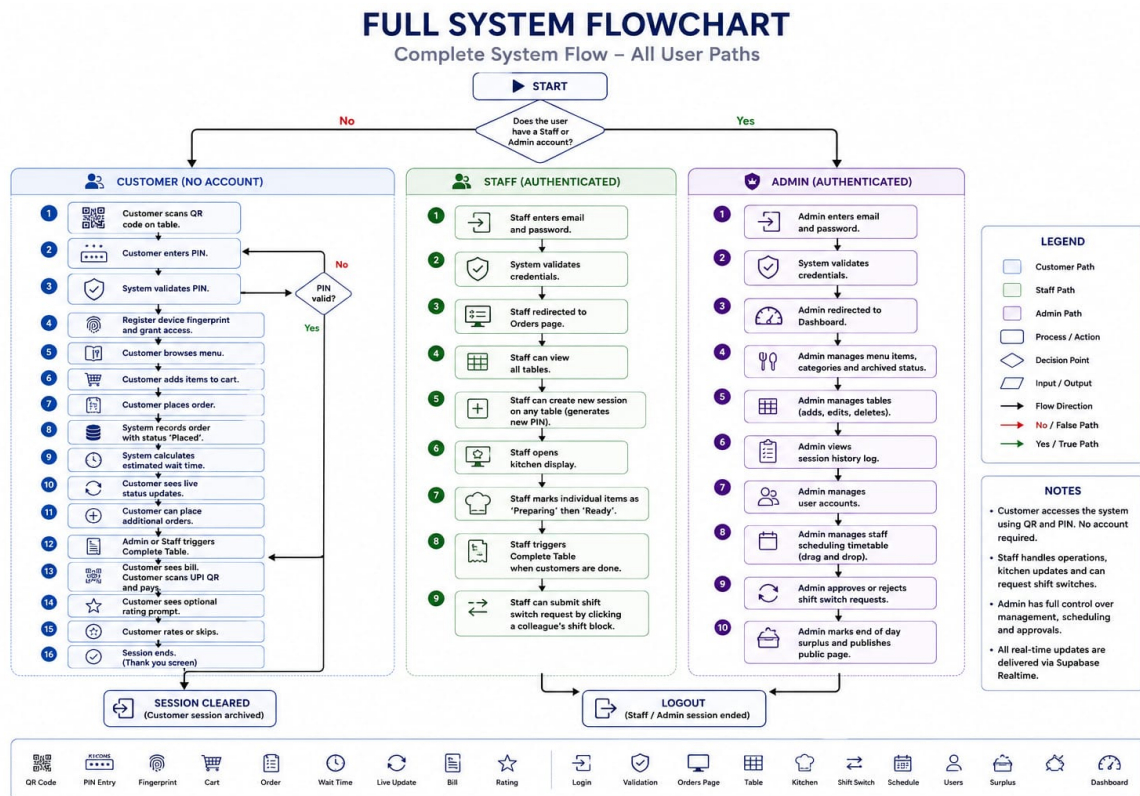
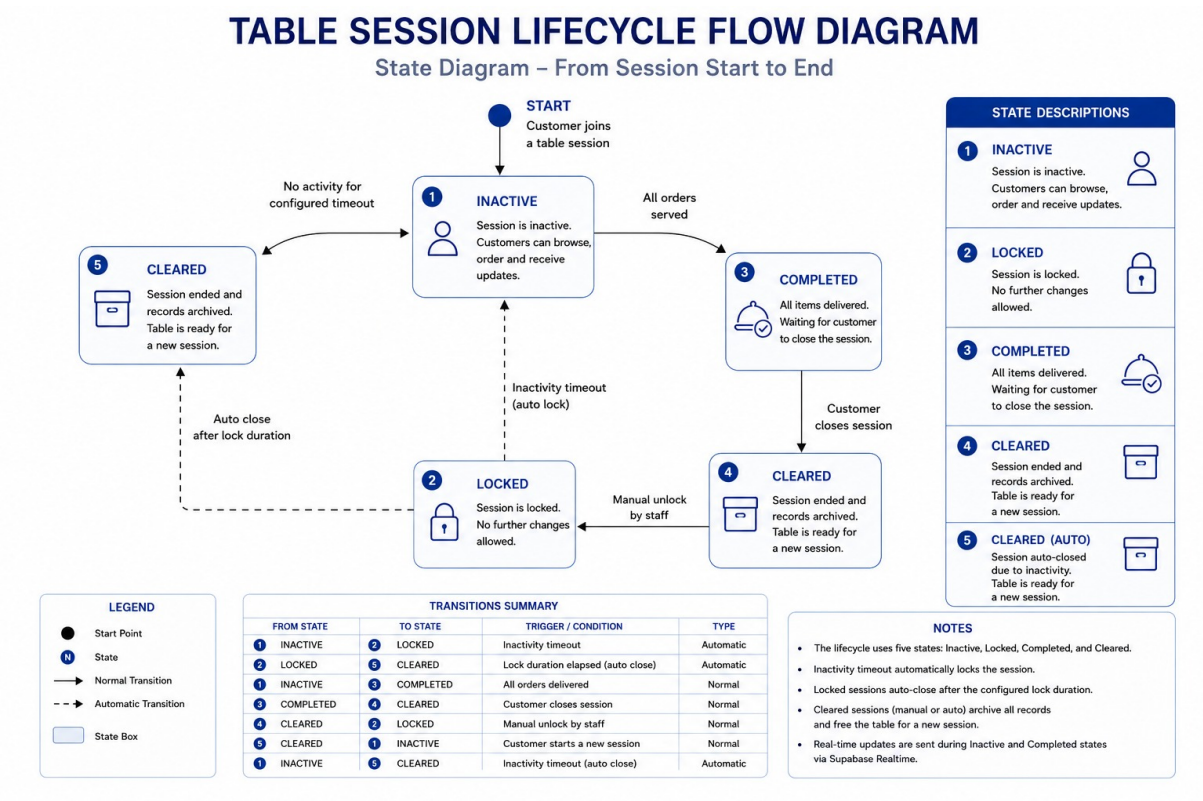


Figure 6: Full System Flowchart





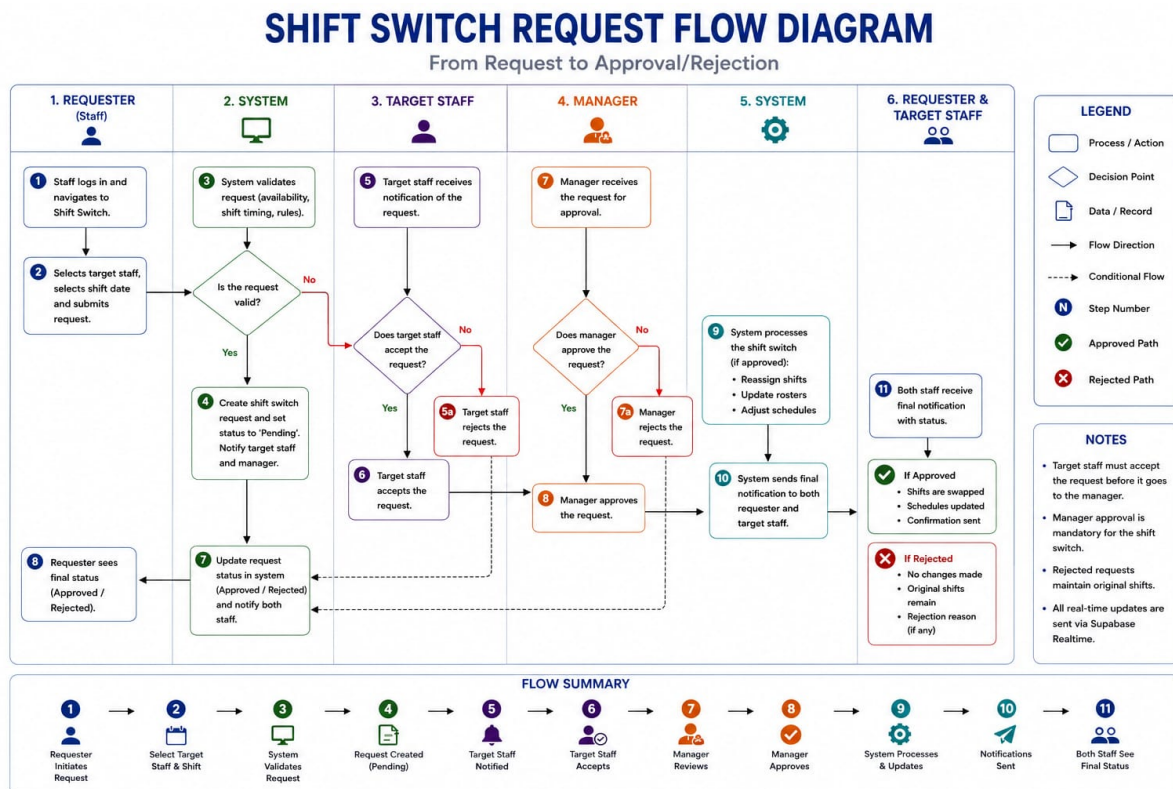


Figure 8: Shift Switch Request Flow Diagram

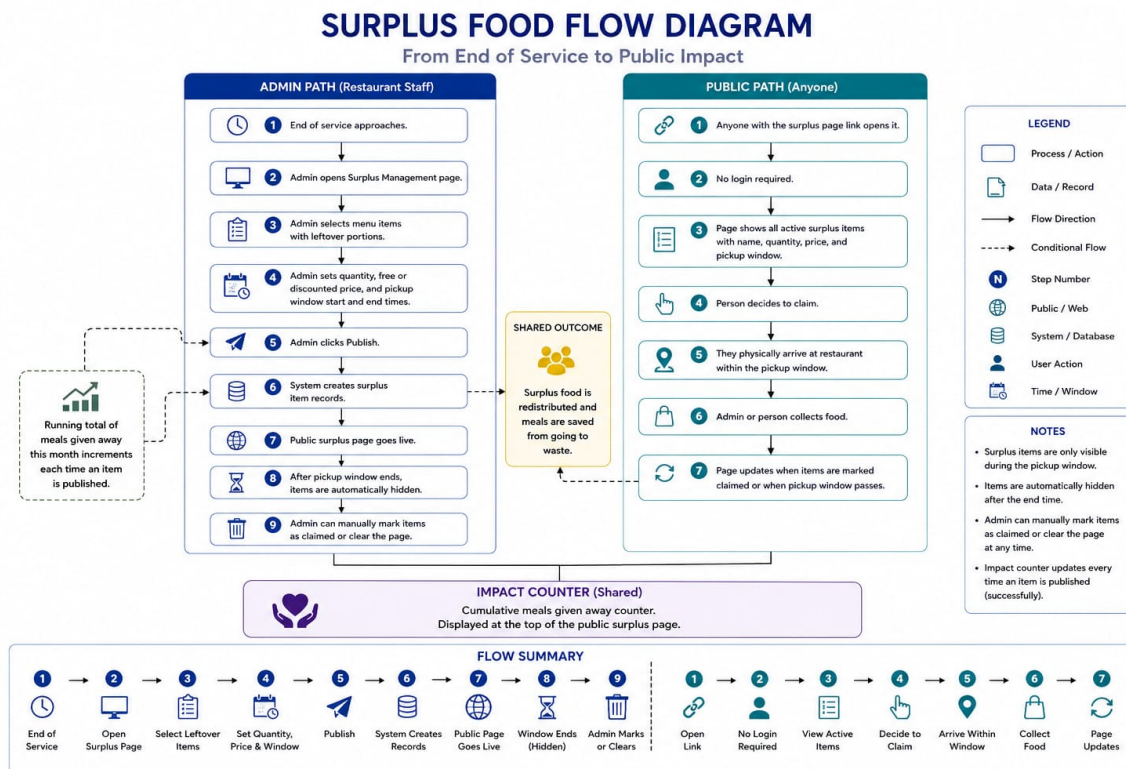


Figure 9: Surplus Food Flow Diagram

## 9 Security

### 9.1 Authentication Security

| Concern          | How it is handled                                                                         |
|------------------|-------------------------------------------------------------------------------------------|
| Password storage | Managed by Supabase Auth. Passwords are hashed automatically, never stored as plain text  |
| Session tokens   | JWT issued by Supabase on login. Stored in browser cookies, not accessible via JavaScript |
| Logout           | Token invalidated server-side. Cookie cleared on client                                   |
| Customer access  | Validated via device fingerprint registered against active session PIN                    |
| Session PIN      | Expires when session is cleared or a new session is created on the same table             |

Table 22: Authentication Security

### 9.2 Authorization — Three Layers

**Layer 1 — Frontend:** Buttons and pages shown or hidden based on role. UX only, not real security.

**Layer 2 — API Routes:** Every sensitive route validates session token and role server-side before executing. Returns 401 if no session, 403 if wrong role.

**Layer 3 — Row Level Security:** Enforced at the Supabase database level. Acts as final line of defence even if API layer is bypassed.

### 9.3 Environment Variables

Supabase keys are stored in `.env.local` locally and in Vercel environment settings in production. `.env.local` is excluded from Git via `.gitignore`.

### 9.4 Known Limitations

- No rate limiting on API routes
- No CAPTCHA on login
- No two-factor authentication
- No audit logging
- UPI payment is display only — payment confirmation is manual
- Surplus food collection is trust-based with no verification

## 10 Future Scope

The following features are outside the current project scope but are natural extensions:

- Ingredient-level inventory tracking with automatic low stock alerts
- Staff payroll and attendance tracking integrated with the scheduling system
- Multi-branch support allowing a restaurant chain to manage all locations from one admin account
- Automated surplus food notifications to registered NGOs or food banks
- Advanced analytics — busiest hours, top selling items, revenue trends
- Native mobile application for Staff and Admin
- Payment gateway integration for direct online payment processing
- Two-factor authentication for Admin accounts
- Printer integration for automatic kitchen order ticket printing